COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010 - Data Structures and Algorithms	**SECTION:**	CPE21S1
DATE PERFORMED:	Oct 13, 2026	**DATE SUBMITTED:**	Oct 15, 2026
NAME:	Nichollo Dave A. Galvez Al Twain D. Pasion	**INSTRUCTOR:**	Engr Jimlord Quejado

ACTIVITY NO. 12 Algorithm Strategies

1. PROCEDURE OUTPUT

What procedures did you perform?

Table 12-1. Algorithmic Strategies and Examples

Strategy	Algorithm	Analysis
Recursion	Fibonacci Algorithm	**GALVEZ:** Fibonacci calculates each term by recursively calling itself to sum the two preceding values. **PASION:** Fibonacci calls itself to sum the current and previous numbers based on the passed parameters.
Brute Force	Bubble Sort Algorithm	**GALVEZ:** Bubble Sort algorithm works by repeatedly stepping through the list to compare adjacent elements, swapping them if they are in the wrong order until no more swapping are needed and the list is fully sorted. **PASION:** Bubble Sort Algorithm has a quadratic time complexity of O(n^2). The reason why it can be considered as a Brute Force strategy is because it sorts the elements one by one, not in the most optimal way.
Backtracking	Insertion Sort Algorithm	**GALVEZ** Insertion sort builds the sorted array one item at a time by taking each element and placing it

		into its correct position relative to the sorted portion. **PASION:** In theory, Insertion Sort, picks the next number after the sorted region of a list, and checks from the highest to lowest, or "backtracks"
Greedy	Selection Sort Algorithm	**GALVEZ:** Selection sort makes a locally optimal choice at each step by selecting the smallest (or largest) element from the unsorted portion of the array and placing it into its final sorted position. **PASION:** It can be considered a Greedy strategy because of how it chooses the smallest/largest element, and never re-evaluates any further of its choice.
Divide-and-Conquer	Shell Sort Algorithm	**GALVEZ:** Shell sort works by breaking an array down into individual elements. From there, it gradually rebuilds the list through progressive stages, identifying and arranging the smallest values from each subgroup into a new order. This recursive cycle continues until all elements are reunited into one sorted array equal to the initial length. **PASION:** Shell sort divides the array into sub-arrays, until it separates all the elements individually. Then it recombines them through stages sequentially, getting the minimum of each sub-group, comparing them, and putting them in order in a newly combined sub-array. This gets repeated through recursion until the original array size has been met.

Table 12-2. Memoization Implementation

Screenshot	
Analysis	**GALVEZ:** So the code is a top down approach, because it breaks down a problem into smaller pieces by using recursion. The getMinSteps function takes int n and breaks it down into smaller subproblems by checking n-1, n/2, and n/3. It also uses a lookup table called memo, to keep the code fast. It checks if(memo[n] != -1) before doing any work so that it will see if the answer was already saved, and if not, it will calculate the result and save it in memo[n] so it never has to repeat the same work twice. **PASION:** For the given program, it uses a top-down approach with “memoization” because it starts from the very main problem and breaks it down into sub problems through recursion. In this scenario, the program starts at the main problem introduced, which is 10. In the function, “getMinSteps”, it splits the problem into subproblems like n-1, n / 2, and n / 3. These calls break down the problem recursively until it hits the very bottom case which is 1. It “caches” these solutions and when a new problem is presented, it’ll first check the cache if it’s already been computed, if not, it will compute it again.

```cpp
#include <iostream>
#include <algorithm>

using namespace std;

//limit value
const int MAX_N = 100;
int memo[MAX_N + 1];

int getMinSteps(int n) {
    if (n == 1) return 0;
    if (memo[n] != -1) return memo[n];

    int r = 1 + getMinSteps(n - 1);
    if (n % 2 == 0) r = min(r, 1 + getMinSteps(n / 2));
    if (n % 3 == 0) r = min(r, 1 + getMinSteps(n / 3));

    memo[n] = r;
    return r;
}

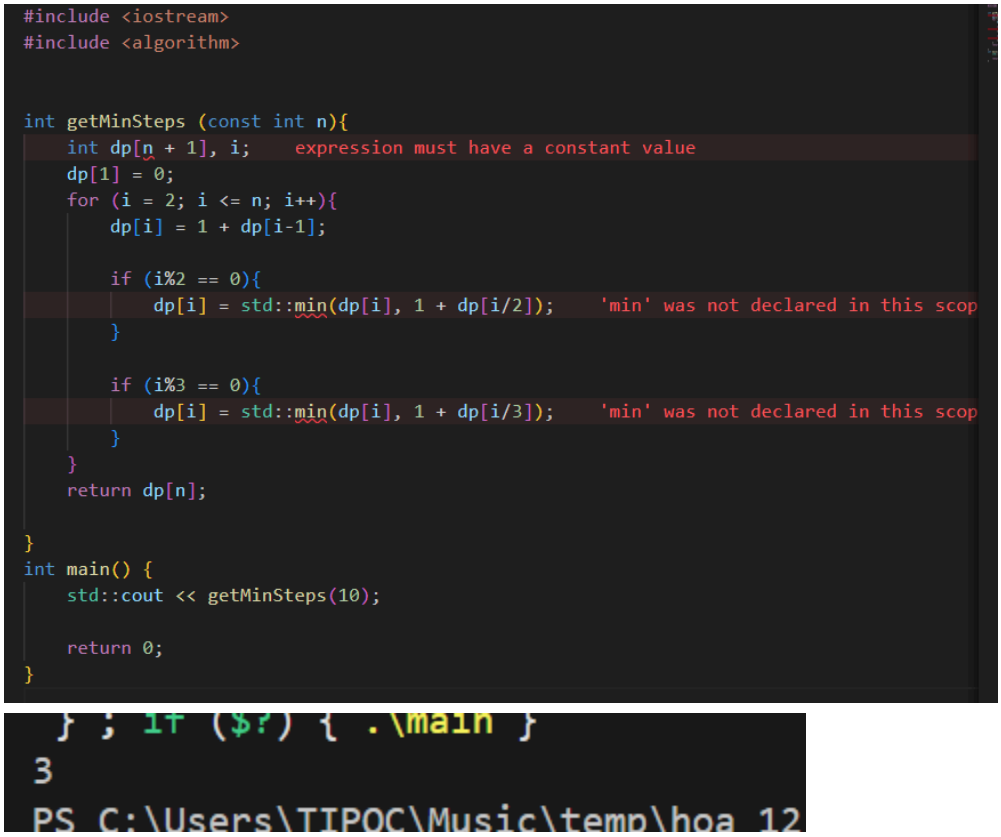
int main() {
    int n = 10;

    fill(memo, memo + MAX_N + 1, -1);

    cout << "Minimum steps for " << n << " is: " << getMinSteps(n) << endl;

    return 0;
}
```

Table 12-3. Bottom-Up Dynamic Programming Implementation

Screenshot	
Analysis	**GALVEZ:** Passing 10 into the function builds an array storing the step counts to reduce numbers 1 to 10 down to 1. It calculates these values using subtraction, division by 2, and division by 3, saving each result in the table. It also checks the entry for 10 to extract the most efficient path to 1. **PASION:** For the bottom-up given code example, we pass the integer 10 through the function. In sequence, the function will first define an array where we can store a “table” of steps for numbers 1 through 10. This array stores how many steps it takes for the numbers to reach 1, in the means of subtracting or dividing. It’ll first do the simplest approach of subtracting until it gets to 1, the second approach first checks if it is divisible by 2, then it’ll divide it until it reaches 1, the last approach first checks if it is divisible by 3, then it’ll divide it until it reaches 1. For every “step” it stores it in the array. Lastly, when it finishes after the number 10, the program will find the most minimum number in the array to get to number 1 from 10 in the most efficient way.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Copy and paste the questions from the lab and include your answers after.

ILO B: Create C++ code to implement various algorithmic strategies & ILO C: Create C++ code to solve problems using learned algorithmic strategies

Problem Title: Count the number of paths in a matrix with a given cost to reach the destination cell

Description:
Given an M × N integer matrix where each cell has a non-negative cost associated with it, count the number of paths to reach the last cell (M-1, N-1) of the matrix from its first cell (0, 0) such that the path has given cost. We can only move one unit right or one unit down from any cell, i.e., from cell (i, j), we can move to (i, j+1) or (i+1, j).

For example, consider the following 4 × 4 matrix where cost is 25. Two paths are having a cost of 25. A sample path has been given when the cost is 25.

4	7	1	6
6	7	3	9
3	8	1	2
7	1	7	3

The idea is to use recursion. The problem has optimal substructure. That means the problem can be broken down into smaller, simple “subproblems”, which can further be divided into yet simpler, smaller subproblems until the solution becomes trivial.

CODE

```cpp
#include <iostream>

int countPath(int i, int j, int currentCost, int targetCost, int grid[4][4], int M, int N) {

    // checks if we are still within the matrix
    if (i >= M || j >= N) {
        return 0;
    }

    // to keep track of the current cost
    int newCost = currentCost + grid[i][j];

    // to end the run early if the new cost is higher than the targetCost
    if (newCost > targetCost) {
        return 0;
    }

    // one we reach the end of the matrix (M-1, N-1)
    if (i == M-1 && j == N-1) {
        if (newCost == targetCost) {
            return 1;
        }
        else {
            return 0;
        }
    }

    // recursion for movements
    int moveRight = countPath(i, j+1,  currentCostnewCost, targetCost, grid, M, N);
    int moveDown = countPath(i+1, j,  currentCostnewCost, targetCost, grid, M, N);

    // to total up all the successful paths that reached the target cost
    return moveRight + moveDown;
}
```

```cpp
int main() {
    int M = 4;
    int N = 4;
    int targetCost = 25;

    int matrix[4][4] = {
        {4, 7, 1, 6},
        {6, 7, 3, 9},
        {3, 8, 1, 2},
        {7, 1, 7, 3}
    };

    int totalPaths = countPath( i: 0, j: 0, currentCost: 0, targetCost, matrix, M, N);

    std::cout << "Number of paths that reached the cost of " << targetCost << " with the given matrix is " << totalPaths << ".\n";

    return 0;
}
```

OUTPUT

```
Number of paths that reached the cost of 25 with the given matrix is 0.

Process finished with exit code 0
```

ANALYSIS

GALVEZ: The main function initializes a 4 x 4 matrix, sets the dimensions M and N to 4, and defines a target cost of 25. It then calls the countPath function, starting at position (0,0) with an initial cost of 0, to search for valid routes through the grid. It prints out the result returned by the function, which in this case displays 0 because no path reaches the exact target cost.

PASION: For this program, we have a function that explores all possible paths from the top-left of the matrix; (0,0), to the very bottom-right; (M-1, N-1) that uses up exactly the target cost. In the function, the first block of code checks if the function has exceeded the matrix or not. The second block is what we use to keep track of the cost of each movement depending on the elements of the matrix. The third block ends the path immediately if we've exceeded the target cost. The fourth block is for only if we reach the bottom-right matrix, and then checking if we've hit exactly the targetCost or less than that. The last block is for when we call the function again when we want to make a movement to the right or downwards continuing where the function is at the matrix currently.. We then return the sum of these movements to add up the number of successful paths that reached the target cost.

CONCLUSION
GALVEZ: In this activity, we explored various algorithmic strategies including Recursion, Brute Force, Backtracking, Greedy, Divide-and-Conquer, and Dynamic Programming. We analyzed how different sorting algorithms fit into these classifications and examined top-down memoization alongside bottom-up dynamic programming implementations. We also applied recursive problem-solving to evaluate paths and target costs within a grid matrix. **PASION:** In this activity, we've learned the five common Strategies in Algorithm; Recursion, Brute Force, Backtracking, Greedy, and Divide-and-Conquer. For the first part of the procedure, me and my partner gave our analysis and insights to different Searching or Sorting algorithms and their classifications of strategy. In the second part of the procedure, we got to understand the top-down and bottom-up coding.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.